

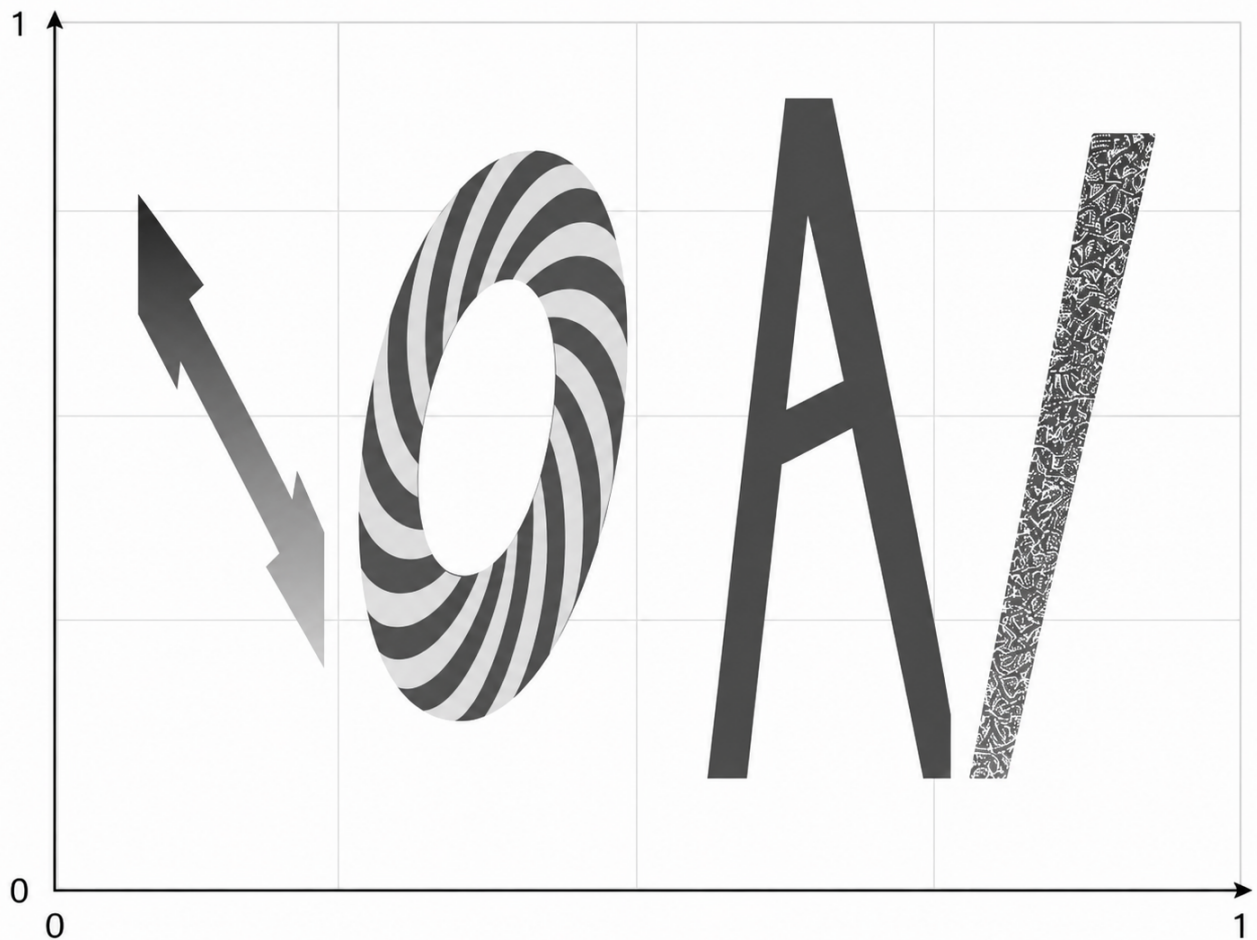
Πεδίο IOAI

- **Χρονικό όριο:** 5 λεπτά
- **Αποθηκευτικός χώρος:** 5 GB
- **Μέγεθος λύσης:** `solution.ipynb`, `custom_model.py` ≤ 1 MB συνολικά
- **Προεκπαιδευμένα μοντέλα:** κανένα — εκπαίδευση από το μηδέν, χωρίς διαδίκτυο κατά τη βαθμολόγηση
- **Βαθμολογία baseline:** 31.2187

Εργασία

Ο Δήμαρχος της Astana θέλει να διακοσμήσει την πόλη με στυλιζαρισμένα λογότυπα IOAI. Ως στατιστικολόγος, αντιλαμβάνεται τα πάντα —συμπεριλαμβανομένου του λογοτύπου— ως μια χωρική συνάρτηση $F(x, y, \overline{W})$, όπου τα $x, y \in [0, 1]$ αναπαριστούν συντεταγμένες σε ένα επίπεδο 2D και το $\overline{W}$ είναι ένα σύνολο κρυφών παραμέτρων που καθορίζουν στιλιστικά χαρακτηριστικά, όπως τα χρώματα και οι γωνίες των γραμμάτων.

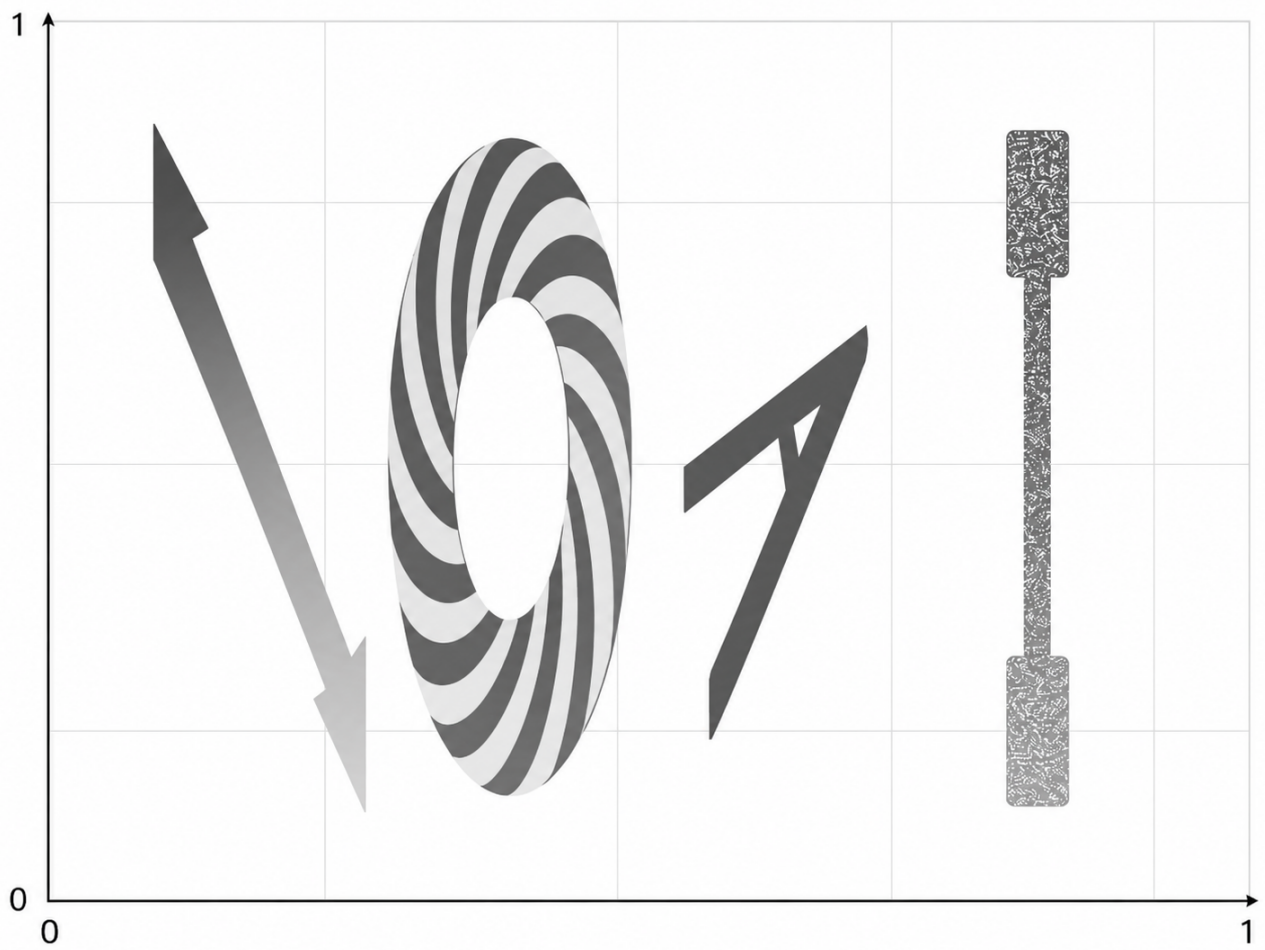
Επειδή η F είναι υπερβολικά σύνθετη για να εκφραστεί ως ρητή μαθηματική εξίσωση, η εργασία σας είναι να εκπαιδεύσετε ένα νευρωνικό δίκτυο ώστε να την προσεγγίζει. Το δίκτυο θα εξάγει μια τιμή **πεδίου IOAI** για οποιοδήποτε ζεύγος συντεταγμένων (x, y) , δημιουργώντας μια πλήρη απεικόνιση θερμικού χάρτη του λογοτύπου σε ολόκληρο το επίπεδο. Ακολουθεί ένα παράδειγμα απεικόνισης θερμικού χάρτη της F με ορισμένες συγκεκριμένες κρυφές παραμέτρους $\overline{W}$.

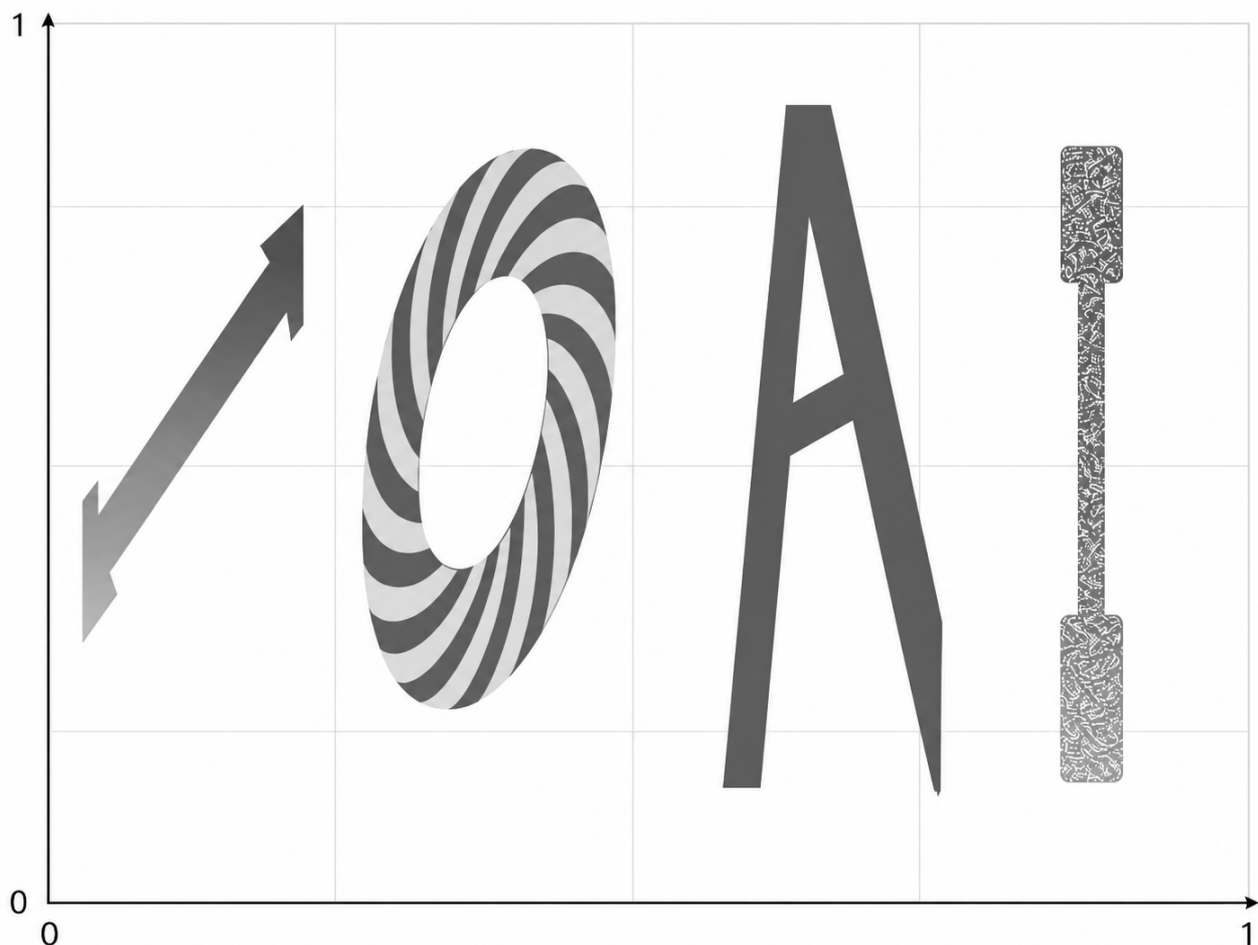


Από τι αποτελείται το πεδίο IOAI; Από τέσσερα γράμματα και το φόντο.

- Οι τιμές μέσα στο πρώτο γράμμα $\mathbb{I}$ είναι πολύ μεγάλες ($1e+10$ και άνω), με γραμμική διαβάθμιση
- Οι τιμές στο γράμμα $\mathbb{O}$ παρουσιάζουν σπειροειδές μοτίβο
- Η τιμή μέσα στο γράμμα $\mathbb{A}$ είναι πάντα -1
- Οι τιμές μέσα στο τελευταίο γράμμα $\mathbb{I}$ πρέπει να είναι τυχαίες τιμές από το εύρος $[-2026, 2026]$, ακόμη και αν υπολογιστούν δύο φορές στο ίδιο σημείο
- Έξω από τα γράμματα, η τιμή είναι πάντα μηδέν

Η συνάρτηση έχει κρυφές παραμέτρους $\overline{W}$, οι οποίες επηρεάζουν την κλίμακα και την κλίση των γραμμών, καθώς και το εύρος τιμών μέσα στο πρώτο γράμμα $\mathbb{I}$. Ωστόσο, τα γράμματα δεν θα τέμνονται. Ακολουθούν μερικά ενδεικτικά παραδείγματα της εμφάνισης του πεδίου IOAI με διαφορετικά $\overline{W}$:





Τι σας παρέχεται:

Αυτό το πρόβλημα ΔΕΝ περιέχει datasets. Αντ' αυτού, σας παρέχεται η συνάρτηση γεννήτριας, η οποία διαμορφώνεται από το αρχείο ρυθμίσεων JSON στο `data/train_config/field_config.json`.

Η διαμόρφωση ελέγχου (test config) είναι κρυφή, αλλά έχει παρόμοια φύση. Η εργασία σας είναι να προσαρμόσετε ένα μοντέλο στη δεδομένη γεννήτρια, χρησιμοποιώντας όσα δεδομένα επιθυμείτε. Οι κατανομές «εκπαίδευσης» και «ελέγχου» σας παράγονται από την ίδια γεννήτρια — απλώς δεν γνωρίζετε σε ποια σημεία (x_i, y_i) θα αξιολογηθείτε.

Η υποβολή σας πρέπει να αποτελείται από:

- την κλάση του μοντέλου εκπαίδευσης αποθηκευμένη ως `custom_model.py`. Αυτό το μοντέλο πρέπει να κληρονομεί από την κλάση `torch.nn.Module` και να χρησιμοποιεί μόνο εισαγωγές `torch`. Πρέπει να περιέχει την κλάση `CustomModel` που χρησιμοποιείται στο notebook `solution.ipynb`.
- το notebook `solution.ipynb`, το οποίο θα παράγει τα βάρη `model.pt`

Βαθμολόγηση

Για κάθε περιοχή, η ελάχιστη βαθμολογία είναι 0 και η μέγιστη βαθμολογία είναι 1. Η τελική βαθμολογία υπολογίζεται ως ο μέσος όρος και των πέντε περιοχών (τέσσερις για κάθε γράμμα και μία για το φόντο) και πολλαπλασιάζεται επί 100. Υπάρχει μια **ποινή παραμέτρων**:

Εάν το μοντέλο σας έχει περισσότερες από 20260 παραμέτρους, η βαθμολογία μειώνεται στο μισό.

Ο αριθμός των παραμέτρων μετριέται μέσω του `sum(p.numel() for p in model.parameters())`. Αναμένουμε το μοντέλο σας να λειτουργεί και σε στοχαστική κατάσταση, με το PyTorch `nn.Dropout` να αποτελεί μέρος του μοντέλου.

Για τις Τυπικές Περιοχές

Για κάθε περιοχή R (πρώτο γράμμα I, O, A, Background), αξιολογούμε το μοντέλο σε $N_R = 512$ σημεία ελέγχου (x_i, y_i) με πραγματικές τιμές v_i και προβλέψεις $\hat{v}_i$. Χρησιμοποιούμε το κανονικοποιημένο Μέσο Απόλυτο Σφάλμα (MAE) ως κύρια μετρική. Το MAE ορίζεται ως:

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

Και η κανονικοποίηση πραγματοποιείται ως

$$\text{score}(R) = 1 - \min\left(\frac{\text{MAE}(R)}{s_R}, 1\right),$$

όπου $s_R > 0$ είναι μια σταθερά κλίμακας.

Για την περιοχή του τελευταίου γράμματος I

Σε αυτήν την περιοχή, **το dropout είναι ενεργοποιημένο κατά την αξιολόγηση**. Για κάθε σημείο ελέγχου j :

1. Εκτελούμε το μοντέλο $K = 10$ φορές για να λάβουμε $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$.
2. Εάν οποιαδήποτε έξοδος βρίσκεται εκτός του εύρους $[-2026, 2026]$, τότε $\text{pointScore}(j) = 0$.
3. Διαφορετικά, υπολογίζουμε την τυπική απόκλιση σ_j των K εξόδων και τη μετατρέπουμε σε βαθμολογία:

$$\text{pointScore}(j) = \min\left(\frac{\sigma_j}{s_E}, 1\right),$$

όπου $s_E > 0$ είναι μια σταθερή σταθερά κλίμακας.

Η βαθμολογία της περιοχής είναι ο μέσος όρος όλων των σημείων της περιοχής:

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

όπου $N_E = K * N_R$.

Με απλά λόγια, όσο μεγαλύτερη ποικιλομορφία έχετε, τόσο μεγαλύτερη θα είναι η βαθμολογία σας για αυτήν την περιοχή. **Δεν μπορείτε να χρησιμοποιήσετε τυχαιότητα σε καθαρή μορφή, συμπεριλαμβανομένων των συναρτήσεων PyTorch `rand*` και `_uniform*` η τυχαιότητα πρέπει να προέρχεται από την εκτέλεση inference με ενεργοποιημένο dropout.**

Τρόπος υποβολής

1. Ανοίξτε το `solution.ipynb` και εκτελέστε όλα τα κελιά.
2. Βελτιώστε το μοντέλο `CustomModel` στο `custom_model.py`
3. Βεβαιωθείτε ότι το τελευταίο σας κελί αποθηκεύει το μοντέλο σας στο αρχείο `model.pt`.
4. Στην καρτέλα Git του JupyterLab, κάντε stage, προσθέστε σχόλιο και κάντε commit τα `solution.ipynb` και `custom_model.py` και, στη συνέχεια, κάντε push.
5. Επιστρέψτε στη σελίδα του Διαγωνισμού και κάντε κλικ στο **Submit**. Το σχόλιο υποβολής πρέπει να είναι το ίδιο με το σχόλιο του προηγούμενου βήματος.